C++ Level 1 — Interview-Ready Handbook

> 40 problems · concepts first, then deep walkthroughs · C++17 Every program in this file is compiled with `g++ -std=c++17 -Wall -Wextra -Wpedantic` and tested.

How to use this handbook

1. Read **Part 0 (Core Concepts)** once. Nearly every problem is a combination of those ideas.
2. For each question, read **How to think** *before* the code, and try to write the solution yourself.
3. Say the **Dry run** out loud. Interviewers care more about your reasoning than about the final code.
4. Finish with **Part 5 (Interview Q&A)** and the **checklists**.

Every solution in this file follows the same template:

Section	Purpose
Problem	What is asked, with examples
Concepts	The ideas you must know
How to think	The reasoning, from brute force to final idea
Solution	Clean, compilable code
Dry run	Step-by-step trace
Edge cases	Inputs that break naive code
Complexity	Time and space
Pitfalls	Mistakes seen in practice
Follow-ups	What the interviewer asks next, with answers

Compile and run

```
g++ -std=c++17 -Wall -Wextra -Wpedantic file.cpp -o program
./program
```

Table of Contents

- **Part 0 — Core Concepts**
- **Part 1 — Basics: I/O, Conditions (Q1–Q8)**
- **Part 2 — Loops and Digit Manipulation (Q9–Q16)**
- **Part 3 — Number Properties (Q17–Q24)**
- **Part 4 — Number Challenges (Q25–Q32) and Patterns (Q33–Q40)**
- **Part 5 — Interview Q&A, Checklists, Concept Map**

PART 0 — CORE CONCEPTS

C1. Program structure and I/O

```
#include <iostream>        // declares cin, cout
using namespace std;       // lets you write cout instead of std::cout

int main() {               // execution starts here
    int x;
    cin >> x;              // read
    cout << x << '\n';     // write
    return 0;              // 0 = success (optional in main)
}
```

- `cin >> a >> b` skips whitespace (spaces, tabs, newlines) between values, so `"12 8"` and `"12\n8"` both work.
- `'\n'` only writes a newline. `endl` writes a newline **and flushes** the buffer (slower in loops).
- **Input failure:** if the user types `abc` for an `int`, `cin` enters a fail state and `x` is set to 0 (C++11). Check with `if (!(cin >> x)) { ... }`.
- **Prompts vs judges:** `cout << "Enter a number: "` is nice interactively, but automated judges compare output exactly. Prompts make you fail. In this handbook the solutions print only the result.
- `using namespace std;` is fine in small programs and interviews; in large codebases it can cause name clashes (for example your own `gcd` versus `std::gcd`).

C2. Data types, ranges, and overflow

Type	Typical size	Range
`int`	32-bit	−2,147,483,648 … 2,147,483,647 (about $\pm 2.1 \times 10^9$)
`long long`	64-bit	−9,223,372,036,854,775,808 … 9,223,372,036,854,775,807 (about $\pm 9.2 \times 10^{18}$)
`unsigned int`	32-bit	0 … 4,294,967,295
`double`	64-bit	about 15–17 significant decimal digits
`char`	8-bit	one character, also a small integer ('0' is 48)

Signed overflow is undefined behavior (UB). It is not "wraps around"; the compiler may assume it never happens and produce surprising code. You must prevent it, not detect it afterwards.

Three safe patterns (used all through the problems):

```
// 1. Widen the type: compute in long long, then range-check.
long long r = (long long)a * b;

// 2. Pre-check addition:  a + b <= MAX   <=>   a <= MAX - b
if (a > LLONG_MAX - b) { /* overflow */ }
```

```
// 3. Pre-check multiplication:  a * b <= MAX   <=>   a <= MAX / b   (b > 0)
if (a > LLONG_MAX / b) { /* overflow */ }
```

Useful constants live in `<climits>`: `INT_MAX`, `INT_MIN`, `LLONG_MAX`, `LLONG_MIN`. Always include it when you use them.

Know these numbers: `20! = 2,432,902,008,176,640,000` is the largest factorial in `long long`. The 93rd Fibonacci number `F(92) = 7,540,113,804,746,346,429` is the last one that fits.

C3. Operators you must get exactly right

Integer division truncates toward zero; `%` takes the sign of the dividend (C++11 and later):

Expression	Value
`7 / 2`	3
`-7 / 2`	-3 (not -4)
`7 % 2`	1
`-7 % 2`	-1 (not 1)
`7 / 2.0`	3.5

Consequences:

- Test oddness with `n % 2 != 0`, never `n % 2 == 1` (fails for negatives).
- `x / 10` drops the last digit and `x % 10` extracts it. Both work for negatives too, which is why `do { ... n /= 10; } while (n != 0)` can count digits of negative numbers without `abs()`.

Logical operators short-circuit: `A && B` does not evaluate `B` if `A` is false. This lets you write `n > 0 && (n & (n-1)) == 0` safely, and `i <= n / i` style guards.

Bitwise operators: `&` AND, `|` OR, `^` XOR, `~` NOT, `<<` shift left (multiply by 2), `>>` shift right (divide by 2).

Typical traps

- `=` assigns, `==` compares: `if (a = 5)` always true and overwrites `a`.
- `&&` binds tighter than `||`: write `(year % 4 == 0 && year % 100 != 0) || year % 400 == 0` with explicit parentheses.
- Integer division before conversion: `double d = 1 / 2;` gives `0`. Use `1.0 / 2`, or `static_cast<double>(a) / b`.

C4. Control flow

```
if (cond1) { ... }
else if (cond2) { ... }
else { ... }

switch (op) {            // only for integral values (int, char, enum)
case '+': ...; break;    // forget `break` and execution falls through to the next case
default: ...;            // everything else
}
```

```
int m = (a > b) ? a : b; // ternary: condition ? value_if_true : value_if_false
```

Order conditions from most specific to most general, and make sure branches are mutually exclusive. Use `>=` where ties must still choose a branch (Q5).

C5. Loops

Loop	Use when	Notes
`for (init; cond; step)`	you know the count or range	counter is scoped to the loop
`while (cond)`	repeat until a condition fails	body may run zero times
`do { } while (cond);`	body must run **at least once**	perfect for digit loops where input `0` is valid

Loop invariant thinking: before writing a loop, state in one sentence what is true at the start of every iteration. Example for Euclid: "`gcd(a, b)` equals the gcd of the original pair."

Off-by-one checklist: `<` vs `<=`, start at 0 or 1, and what happens for `n = 0`, `1`, `2`.

Infinite-loop trap: `for (int n = start; n <= end; n++)` never ends if `end == INT_MAX` (the increment overflows). The safe form is `for (int n = start;; n++) { ...; if (n == end) break; }` (Q27).

C6. Functions, references, and return values

```
void swapNumbers(int &a, int &b) { int t = a; a = b; b = t; }  // & = reference: changes the calle

bool reverseInteger(int num, int &result);   // return success/failure AND an output value
```

- **Pass by value** copies the argument; changes stay inside the function.
- **Pass by reference (`&`)** works on the original variable.
- Returning `bool` for success plus an output parameter is a clean way to report overflow (Q10).
- Break programs into small functions (`isPrime`, `gcd`): they are easier to test and to explain.

C7. The digit toolkit (the most reused idea in this handbook)

Goal	Code	Meaning
last digit	`n % 10`	remainder when dividing by 10
remove last digit	`n /= 10`	integer division
append digit `d`	`r = r * 10 + d`	shift left in base 10
loop over all digits	`while (n > 0) { d = n % 10; n /= 10; }`	right to left
count digits	`do { count++; n /= 10; } while (n != 0);`	
first digit	`while (n >= 10) n /= 10;`	

The same pattern works for any base: replace `10` with `2` for binary (Q29, Q30).

C8. Number theory essentials

1. **Divisors come in pairs** `(i, n / i)`. If `i` is a divisor, so is `n / i`. One of the two is always ≤ √n, so scanning to √n finds everything.
2. **Prime:** an integer > 1 with no divisors other than 1 and itself. Remember: 1 is not prime, 2 is the only even prime.
3. **Every prime greater than 3 has the form 6k ± 1**, which is why the fast test checks `i` and `i + 2` with step 6.
4. **Euclid:** `gcd(a, b) = gcd(b, a % b)`, and `gcd(a, 0) = a`. Then `lcm(a, b) = a / gcd(a, b) * b`.
5. **Fundamental theorem:** every integer > 1 has a unique prime factorization.
6. **Avoid** `i * i <= n` (can overflow); write `i <= n / i`.

C9. Bit basics

- A number in binary is a sum of powers of two: `13 = 1101₂ = 8 + 4 + 0 + 1`.
- `n & 1` is the last bit (1 means odd).
- `n >> 1` equals `n / 2` for non-negative `n`; `n << 1` equals `n * 2`.
- `n & (n - 1)` clears the lowest set bit. Zero result means at most one bit was set.
- `x ^ x == 0` and `x ^ 0 == x`; XOR is its own inverse (used in XOR swap).

C10. Characters and strings (minimal)

- Digit character to number: `'7' - '0' == 7`. Number to character: `char('0' + 7) == '7'`.
- Build a string with `+=` and flip it with `reverse(s.begin(), s.end())` from `<algorithm>`.
- Read a long digit string with `string s; cin >> s;` when the number may not fit in any integer type (Q30).

C11. Pattern-printing method (Q33–Q40)

1. Outer loop = **rows** `i`. Inner loop(s) = what is printed **in that row**.
2. Make a table: for row `i`, how many spaces? how many symbols? what values?
3. Turn the table into formulas (`i`, `n - i`, `2i - 1`).
4. Print separators **between** items, never after the last one.

C12. Complexity cheat sheet

Growth	Example in this handbook
O(1)	power of two via bits, digital root formula, sum formula
O(log n)	counting digits, Euclid's gcd, decimal-to-binary
O(√n)	prime test, perfect number, factorization
O(n)	factorial, Fibonacci, sum 1..N loop, multiplication table
$O(n^2)$	pattern printing (rows × columns)

Rule of thumb: about 10^8 simple operations per second. O(n) is fine up to ~10^8, O(√n) up to ~10^{14}, O(n^2) only up to ~10^4.

C13. Edge-case checklist (use on every problem)

`0` · `1` · `2` · negative numbers · the maximum value of the type · equal inputs · very large inputs · repeated digits · trailing zeros (`1200`) · single-digit numbers · empty or invalid input.

PART 1 — BASICS: INPUT/OUTPUT AND CONDITIONS

Q1. Print Hello, World!

Problem: Print `Hello, World!` on the screen.

Concepts: program structure, `cout`, exact output matching.

How to think: There is no logic; the point is to know the skeleton and to match the text **exactly** (capital `W`, comma, exclamation mark).

Solution

```
#include <iostream>

int main() {
    std::cout << "Hello, World!" << std::endl;
    return 0;
}
```

Edge cases: none, but a missing comma or wrong capital is a failed test.

Complexity: O(1).

Pitfalls: `Hello world!` vs `Hello, World!`; forgetting `#include <iostream>`; using `cout` without `std::` or `using namespace std;`.

Follow-ups

- `endl` *vs* `'\n'`*?* `endl` writes a newline and flushes the stream; `'\n'` only writes a newline and is faster in loops.
- *Is* `return 0;` *required?* Not in `main` (the compiler adds it), but it documents success.
- *What does* `std::` *mean?* It's the standard library namespace.

Q2. Sum of Two Integers

Problem: Read two integers and print their sum. `Input: 12 8` → `Output: 20`.

Concepts: variables, `cin`, integer overflow (C2).

How to think: Read, add, print. The only trap is the type: two `int`s near $2 \cdot 10^9$ add up to more than `INT_MAX`, which is undefined behavior. Use `long long`.

Solution

```cpp
#include <iostream>
using namespace std;

int main() {
    long long a, b;
    cin >> a >> b;
    cout << a + b << '\n';
    return 0;
}
```

Dry run: `a = 12, b = 8` → `12 + 8 = 20`.

Input	Output
`12 8`	`20`
`-5 5`	`0`
`2000000000 2000000000`	`4000000000` (would overflow `int`)

Complexity: O(1).

Follow-ups

- *What if numbers go up to 10^{18}?* The sum can reach $2 \cdot 10^{18}$, still inside `long long`. For $9 \cdot 10^{18} + 9 \cdot 10^{18}$ you need an overflow check or `__int128`.
- *What does* `cin >> a >> b` *do with* `"12\n8"`*?* Whitespace including newlines is skipped, so it still works.

Q3. Even or Odd

Problem: Read an integer and say whether it is even or odd. `17` → `Odd`.

Concepts: `%` operator, sign behavior (C3), `if/else`.

How to think: Even means divisible by 2, so the remainder is 0. For negatives, `-3 % 2 == -1`, so test `== 0` for even and treat everything else as odd.

Solution

```cpp
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;
    cout << (n % 2 == 0 ? "Even" : "Odd") << '\n';
    return 0;
}
```

Edge cases

Input	Output	Note
`0`	Even	0 % 2 == 0
`-3`	Odd	`-3 % 2` is `-1`, not `0`
`17`	Odd	

Complexity: O(1).

Pitfalls: Writing `n % 2 == 1` for "odd" gives the wrong answer for negative odd numbers.

Follow-ups

- *Another way without `%`?* `n & 1` is 1 for odd, 0 for even. It works for negatives in two's complement.
- *Which is faster?* Compilers produce the same code for both; choose the clearer one.

Q4. Largest of Two Numbers

Problem: Read two numbers and print the larger one. Handle equal numbers. `9 9` → `Equal`.

Concepts: comparison, `if / else if / else`, mutually exclusive branches.

How to think: Three possible outcomes: `a > b`, `b > a`, or neither (equal). Handle the equal case explicitly because the problem asks for it.

Solution

```
#include <iostream>
using namespace std;

int main() {
    long long a, b;
    cin >> a >> b;

    if (a > b)      cout << a << '\n';
    else if (b > a) cout << b << '\n';
    else            cout << "Equal\n";

    return 0;
}
```

Input	Output
`3 8`	`8`
`9 9`	`Equal`
`-1 -5`	`-1`

Complexity: O(1).

Follow-ups

- *Shortest way?* `max(a, b)` from `<algorithm>`, or `a > b ? a : b`.

- *How would you handle* `double` *inputs?* Same code; but comparing doubles with `==` is risky (use a tolerance).

Q5. Largest of Three Numbers

Problem: Find the largest of three integers without a sorting function. `14 27 19` → `27`.

Concepts: compound conditions (`&&`), ties, running maximum.

How to think: *Approach 1:* test each number against the other two with `>=`. *Approach 2 (scales better):* keep a running maximum.

Solution (running maximum)

```
#include <iostream>
using namespace std;

int main() {
    int a, b, c;
    cin >> a >> b >> c;

    int largest = a;
    if (b > largest) largest = b;
    if (c > largest) largest = c;

    cout << largest << '\n';
    return 0;
}
```

Solution (direct comparison, shows tie handling)

```
if (a >= b && a >= c)      cout << a << '\n';
else if (b >= a && b >= c) cout << b << '\n';
else                       cout << c << '\n';
```

Why `>=`? With `5 5 3`, `a >= b && a >= c` is true so `5` is printed. With `>` no branch would match when two values tie for the max, and the final `else` would wrongly print `c`.

Input	Output
`14 27 19`	`27`
`5 5 3`	`5`
`7 7 7`	`7`
`-1 -2 -3`	`-1`

Complexity: O(1) for 3 values, O(n) for n values using the running maximum.

Follow-ups

- *Generalize to n numbers?* Same running-maximum loop; initialize `largest` with the first element, **not** with 0 (all-negative input would break).

- *Number of comparisons?* The running maximum needs `n - 1`.

Q6. Simple Calculator

Problem: Read two numbers and an operator (`+ - * /`), print the result, handle division by zero. `10 2 /` → `5`.

Concepts: `switch`, `char`, `double`, division by zero.

How to think: Read in the order the example gives (numbers first, then operator). Use `double` so decimals work. Dispatch on the operator with `switch`. Guard `/` explicitly, since `double / 0` produces `inf` and integer `/ 0` is undefined behavior.

Solution

```
#include <iostream>
using namespace std;

int main() {
    double a, b;
    char op;
    cin >> a >> b >> op;

    switch (op) {
    case '+': cout << a + b << '\n'; break;
    case '-': cout << a - b << '\n'; break;
    case '*': cout << a * b << '\n'; break;
    case '/':
        if (b == 0) cout << "Error: division by zero\n";
        else        cout << a / b << '\n';
        break;
    default:
        cout << "Invalid operator\n";
    }
    return 0;
}
```

Input	Output
`10 2 /`	`5`
`7 0 /`	`Error: division by zero`
`2.5 2 *`	`5`
`3 4 %`	`Invalid operator`

Complexity: O(1).

Pitfalls

- Forgetting `break` makes execution fall through into the next case.
- With `int a, b`, `7 / 2` prints `3`. Use `double` (or cast) for real division.

- `cout` prints doubles with 6 significant digits by default. For more, use `setprecision` from `<iomanip>`.

Follow-ups

- *Why* `switch` *and not* `if/else`*?* Cleaner for many discrete values; the compiler can build a jump table.
- *How do you add* `%`*?* Only for integers: `a % b` with a zero check. `double` needs `fmod`.

Q7. Leap Year

Problem: Decide whether a year is a leap year (Gregorian rules). `2000` → `Leap year`.

Concepts: boolean logic, `&&`, `||`, precedence.

How to think: The rule has an exception to an exception:

1. Divisible by 4 → leap, **except**
2. divisible by 100 → not leap, **except**
3. divisible by 400 → leap.

In one expression: `(y % 4 == 0 && y % 100 != 0) || y % 400 == 0`.

Solution

```
#include <iostream>
using namespace std;

int main() {
    int year;
    cin >> year;

    bool leap = (year % 4 == 0 && year % 100 != 0) || year % 400 == 0;
    cout << (leap ? "Leap year" : "Not leap year") << '\n';
    return 0;
}
```

Year	Result	Reason
2000	Leap	divisible by 400
1900	Not leap	divisible by 100 but not 400
2024	Leap	divisible by 4, not 100
2023	Not leap	not divisible by 4

Complexity: O(1).

Follow-ups

- *Why does the rule exist?* The solar year is about 365.2425 days; the 4/100/400 rule averages exactly 365.2425.
- *Common wrong answer?* Only checking `year % 4 == 0` fails for 1900 and 2100.

Q8. Multiplication Table

Problem: Print the multiplication table of a number from 1 to 10. `5` → `5 10 15 ... 50`.

Concepts: `for` loop, loop counter, formatting.

How to think: Repeat the same line ten times with a changing multiplier. A `for` loop from 1 to 10 inclusive does that.

Solution

```
#include <iostream>
using namespace std;

int main() {
    long long n;
    cin >> n;

    for (int i = 1; i <= 10; i++)
        cout << n << " x " << i << " = " << n * i << '\n';

    return 0;
}
```

Output for `5`:

```
5 x 1 = 5
5 x 2 = 10
...
5 x 10 = 50
```

If the judge wants just the numbers (`5 10 15 ... 50`), change the loop body to print `n * i` with a space between values:

```
for (int i = 1; i <= 10; i++) {
    if (i > 1) cout << ' ';
    cout << n * i;
}
cout << '\n';
```

Complexity: O(1) (always 10 iterations).

Pitfalls: `i < 10` stops at 9 (off-by-one); trailing space at the end of a line.

Follow-ups

- *Print a table up to N instead of 10?* Replace `10` with a variable; the complexity becomes O(N).
- *What if* `n` *is huge?* `n * i` can overflow; use `long long` or a check.

PART 2 — LOOPS AND DIGIT MANIPULATION

Q9. Sum of First N Natural Numbers

Problem: Read `n` and compute `1 + 2 + … + n` using a loop. `5` → `15`.

Concepts: accumulator pattern, loop, overflow, closed-form formula.

How to think

1. **Brute force:** `sum = 0`, add `1..n` one by one. O(n).
2. **Optimization (Gauss):** pair the first and last terms: `1+n`, `2+(n-1)`, … There are `n/2` pairs, each summing to `n + 1`, so `sum = n(n+1)/2`. O(1).
3. **Overflow:** `n * (n + 1)` can overflow before the division. Divide the even factor first.

Solution

```
#include <iostream>
using namespace std;

int main() {
    long long n;
    cin >> n;

    if (n < 0) {
        cout << "Enter a non-negative number\n";
        return 0;
    }

    long long sum = 0;                    // loop version, O(n)
    for (long long i = 1; i <= n; i++)
        sum += i;

    long long formula = (n % 2 == 0)      // formula version, O(1)
                        ? (n / 2) * (n + 1)
                        : n * ((n + 1) / 2);

    cout << sum << '\n' << formula << '\n';
    return 0;
}
```

Dry run (n = 5): `sum`: 0 → 1 → 3 → 6 → 10 → 15. Formula: `n` odd → `5 * (6/2) = 15`. ✓

Input	Output
`5`	`15`
`0`	`0`
`100`	`5050`

Complexity: loop O(n) time, formula O(1). Both O(1) space.

Pitfalls: starting `sum` uninitialized; using `int` for the sum; `i < n` instead of `i <= n`.

Follow-ups

- *Why divide the even factor first?* `n * (n+1)` is always even, but dividing **after** multiplying risks overflow even when the final answer fits.

- *Proof of the formula?* Write the sum forwards and backwards and add: every column adds to `n + 1`, `n` columns, so `2·sum = n(n+1)`.

Q10. Reverse an Integer

Problem: Reverse the digits of an integer without converting to a string. Detect overflow. `1234` → `4321`.

Concepts: digit toolkit (C7), widening to avoid overflow, `bool` + reference return.

How to think: Peel digits from the right with `% 10` and push them onto the result with `result * 10 + digit`. The reversed value of a 10-digit `int` can reach 9,999,999,999, bigger than `INT_MAX`, so compute in `long long` and check the range at the end.

Solution

```
#include <iostream>
#include <climits>
using namespace std;

bool reverseInteger(int num, int &result) {
    long long n = num;                      // widen first: -INT_MIN is safe in long long
    int sign = (n < 0) ? -1 : 1;
    if (n < 0) n = -n;

    long long reversed = 0;
    while (n > 0) {
        reversed = reversed * 10 + n % 10;   // push the last digit of n
        n /= 10;                             // drop it from n
    }
    reversed *= sign;

    if (reversed < INT_MIN || reversed > INT_MAX)
        return false;                        // does not fit in int

    result = static_cast<int>(reversed);
    return true;
}

int main() {
    int num, result;
    cin >> num;
    if (reverseInteger(num, result)) cout << result << '\n';
    else                             cout << "Overflow\n";
    return 0;
}
```

Dry run (1234):

n	n % 10	reversed
1234	4	4
123	3	43
12	2	432

n	n % 10	reversed
1	1	4321
0	loop ends	**4321**

Input	Output	Note
`1200`	`21`	trailing zeros vanish
`0`	`0`	loop skipped
`1534236469`	`Overflow`	reverse is 9646324351
`-123`	`-321`	sign handled

Complexity: O($\log_{10} n$) time (one step per digit), O(1) space.

Pitfalls: doing `reversed` in `int` (overflows silently, undefined behavior); forgetting that `-INT_MIN` overflows `int`.

Follow-ups

- *How do you detect overflow without* `long long`*?* Before `rev = rev * 10 + d`: if `rev > INT_MAX / 10` or (`rev == INT_MAX / 10` and `d > 7`), report overflow.
- *Why does* `1200` *give* `21`*?* The digits are `0,0,2,1`; leading zeros of a number are not stored.

Q11. Count the Digits

Problem: Count the digits of an integer using arithmetic. `0` has one digit. Bonus: negatives, without `abs()`.

Concepts: `do-while`, truncating division by 10, negatives.

How to think: Each division by 10 removes one digit; count the divisions until the number becomes 0. The input `0` must still count once, so the body must run **before** the test: `do-while`.

Solution

```
#include <iostream>
using namespace std;

int main() {
    long long n;
    cin >> n;

    int count = 0;
    do {
        count++;
        n /= 10;
    } while (n != 0);              // != 0 (not > 0) so negatives work too

    cout << count << '\n';
```

```
    return 0;
}
```

Dry run (-987): count 1, n = -98 → count 2, n = -9 → count 3, n = 0 → stop. Output `3`.

Why negatives work: C++ integer division truncates toward zero, so `-98 / 10 = -9` and `-9 / 10 = 0`. No `abs()` needed (and `abs(INT_MIN)` would overflow anyway).

Input	Output
`12345`	`5`
`7`	`1`
`0`	`1`
`-987`	`3`

Complexity: O($\log_{10} n$) time, O(1) space.

Pitfalls: `while (n > 0)` returns `0` for input `0`; `int` is enough for 10 digits, but `long long` keeps larger inputs safe.

Follow-ups

- *Count digits with logarithms?* `floor(log10(n)) + 1` for `n > 0`, but floating-point error can bite near powers of 10; the loop is exact.

Q12. Check for a Prime Number

Problem: Decide whether an integer is prime. Handle numbers < 2 and avoid testing every divisor up to `n - 1`.

Concepts: primes, √n bound, the 6k ± 1 form (C8), overflow-safe loop condition.

How to think

1. **Brute force:** test `2 .. n-1`. O(n).
2. **√n bound:** if `n = a × b` is composite, one of `a, b` is ≤ √n. So test only up to √n. O(√n).
3. **Skip obvious multiples:** after excluding 2 and 3, all primes are `6k ± 1`, so test `i` and `i + 2` with step 6 (about 3× fewer checks).

Solution

```
#include <iostream>
using namespace std;

bool isPrime(int n) {
    if (n <= 1) return false;                // 0, 1, negatives
    if (n <= 3) return true;                 // 2 and 3
    if (n % 2 == 0 || n % 3 == 0) return false;

    for (int i = 5; i <= n / i; i += 6)      // i * i <= n, written overflow-safe
        if (n % i == 0 || n % (i + 2) == 0)
            return false;
```

```
    return true;
}

int main() {
    int n;
    cin >> n;
    cout << (isPrime(n) ? "Prime" : "Not prime") << '\n';
    return 0;
}
```

Dry run (n = 49): not ≤ 3, 49 % 2 = 1, 49 % 3 = 1. `i = 5`: `5 <= 49/5 = 9` → 49 % 5 = 4, 49 % 7 = 0 → **Not prime**.

Input	Output
`2`, `3`	Prime
`1`, `0`, `-7`	Not prime
`49`	Not prime (7 × 7)
`29`	Prime
`2147483647`	Prime (a Mersenne prime)

Complexity: O(√n) time, O(1) space.

Pitfalls: declaring 1 prime; treating 2 as non-prime because `2 % 2 == 0`; `i * i <= n` overflowing for `n` near `INT_MAX`.

Follow-ups

- *Why is 6k ± 1 enough?* Any integer is `6k + r`, `r ∈ {0..5}`. `6k, 6k+2, 6k+4` are even, `6k+3` is divisible by 3, leaving only `6k+1` and `6k+5 = 6(k+1) − 1`.
- *Test many numbers?* Use the Sieve of Eratosthenes (see Q27).

Q13. Factorial of a Number

Problem: Compute `n!` with a loop. Handle `0! = 1` and overflow.

Concepts: product accumulator, overflow pre-check (C2).

How to think: `n! = 1 × 2 × … × n`. Start the accumulator at 1 (the multiplicative identity), which handles `0!` and `1!` for free because the loop never runs. Factorials grow extremely fast: `21!` already exceeds `long long`. Check **before** multiplying: `fact * i <= LLONG_MAX` ⟺ `fact <= LLONG_MAX / i`.

Solution

```
#include <iostream>
#include <climits>
using namespace std;

int main() {
    int n;
    cin >> n;
```

```
    if (n < 0) {
        cout << "Factorial is undefined for negative numbers\n";
        return 0;
    }

    long long fact = 1;
    for (int i = 2; i <= n; i++) {
        if (fact > LLONG_MAX / i) {          // fact * i would overflow
            cout << "Overflow\n";
            return 0;
        }
        fact *= i;
    }
    cout << fact << '\n';
    return 0;
}
```

Dry run (n = 5): 1 → ×2 = 2 → ×3 = 6 → ×4 = 24 → ×5 = **120**.

Input	Output
`0`, `1`	`1`
`5`	`120`
`20`	`2432902008176640000`
`21`	`Overflow`
`-3`	undefined message

Complexity: O(n) time, O(1) space.

Pitfalls: starting `fact` at 0; multiplying first and testing afterwards (already undefined behavior); using `int` (overflows at 13!).

Follow-ups

- *Why does* `0! = 1`*?* It is the empty product, and it makes `n! = n × (n−1)!` and combinatorics formulas consistent.
- *Need bigger factorials?* Use big-integer arithmetic (array of digits) or modular arithmetic.

Q14. Fibonacci Sequence

Problem: Print the first `n` Fibonacci numbers using iteration. `6` → `0 1 1 2 3 5`.

Concepts: two-variable rolling state, iteration vs recursion, overflow.

How to think: Each term is the sum of the previous two. Only the last two values matter, so keep two variables and slide them forward. This is O(n) time and O(1) space; the naive recursive `fib(n-1) + fib(n-2)` is exponential.

Solution

```cpp
#include <iostream>
#include <climits>
using namespace std;

int main() {
    int n;
    cin >> n;

    if (n < 0) {
        cout << "Enter a non-negative number\n";
        return 0;
    }

    long long a = 0, b = 1;                  // F(0), F(1)
    for (int i = 0; i < n; i++) {
        if (i > 0) cout << ' ';
        cout << a;

        if (i < n - 1) {                     // don't compute a term we won't print
            if (a > LLONG_MAX - b) {         // a + b would overflow
                cout << "\nOverflow\n";
                return 0;
            }
            long long next = a + b;
            a = b;
            b = next;
        }
    }
    cout << '\n';
    return 0;
}
```

Dry run (n = 6):

i	print a	next = a + b	(a, b) after
0	0	1	(1, 1)
1	1	2	(1, 2)
2	1	3	(2, 3)
3	2	5	(3, 5)
4	3	8	(5, 8)
5	5	skipped	

Output: `0 1 1 2 3 5`.

Input	Output
`0`	(empty line)
`1`	`0`

Input	Output
`2`	`0 1`
`92`	prints all 92 terms (largest is F(91) = 4660046610375530309)
`93` or more	prints 92 terms, then `Overflow` (the loop computes one look-ahead term, F(93), which exceeds `long long`)

Complexity: O(n) time, O(1) space.

Pitfalls: off-by-one in how many terms print; trailing space; recursion for large `n`.

Follow-ups

- *Recursive version complexity?* $O(2^n)$; memoization makes it O(n).
- *Fast Fibonacci?* Matrix exponentiation or fast doubling gives O(log n).
- *Why check overflow before the addition?* After a signed overflow the behavior is already undefined.

Q15. Sum of Digits

Problem: Sum all digits of an integer. `583` → `16`.

Concepts: digit toolkit, accumulator, `do-while`.

How to think: Take `n % 10`, add it, drop it with `n /= 10`, repeat. Use the absolute value so negatives work, and `do-while` so `0` is handled.

Solution

```
#include <iostream>
using namespace std;

int main() {
    long long n;
    cin >> n;

    if (n < 0) n = -n;

    int sum = 0;
    do {
        sum += static_cast<int>(n % 10);
        n /= 10;
    } while (n > 0);

    cout << sum << '\n';
    return 0;
}
```

Dry run (583): `3` → sum 3; `8` → 11; `5` → **16**.

Input	Output
`583`	`16`

Input	Output
`0`	`0`
`-45`	`9`
`1000`	`1`

Complexity: O($\log_{10}$ n) time, O(1) space.

Follow-ups

- *Where else does this idea appear?* Digital root (Q32), divisibility by 3 or 9 (a number is divisible by 9 iff its digit sum is), Armstrong and Strong numbers.

Q16. Armstrong Number

Problem: Check whether a non-negative integer equals the sum of its digits each raised to the number of digits. `153` → Armstrong.

Concepts: two passes over digits, integer power without `pow`, overflow guard.

How to think

1. **Pass 1:** count digits `d`.
2. **Pass 2:** sum `digit^d` for every digit.
3. Compare with the original. Use an integer power loop; `std::pow` returns `double` and can give `124.99999` that truncates to 124.
4. For 19-digit inputs the sum could overflow, but the sum only grows, so stop as soon as it exceeds `n`.

Solution

```
#include <iostream>
using namespace std;

long long integerPower(int base, int exp) {
    long long result = 1;
    while (exp-- > 0)
        result *= base;
    return result;
}

int main() {
    long long n;
    cin >> n;

    if (n < 0) {
        cout << "Not an Armstrong number\n";
        return 0;
    }

    long long temp = n;                         // pass 1: count digits
    int digits = 0;
    do {
```

```
        digits++;
        temp /= 10;
    } while (temp > 0);

    temp = n;                                   // pass 2: sum of powers
    long long sum = 0;
    do {
        int digit = static_cast<int>(temp % 10);
        sum += integerPower(digit, digits);
        if (sum > n) break;                     // can never come back down
        temp /= 10;
    } while (temp > 0);

    cout << (sum == n ? "Armstrong number" : "Not an Armstrong number") << '\n';
    return 0;
}
```

Dry run (153): `digits = 3`; `3³ = 27`, `5³ = 125`, `1³ = 1` → `153` ✓.

Input	Output
`153`, `370`, `371`, `407`	Armstrong (3-digit ones)
`9474`	Armstrong ($9^4 + 4^4 + 7^4 + 4^4$)
`0` – `9`	all Armstrong ($d^1 = d$)
`123`	Not

Complexity: $O(d^2)$ worst case (d digits, each power costs O(d)); trivial for 64-bit inputs.

Pitfalls: hard-coding the exponent as 3; `pow` with floating-point rounding; counting digits of `0` as zero.

Follow-ups

- *Same idea, other names?* "Narcissistic numbers" or pluperfect digital invariants.
- *Faster?* Precompute `d^k` for digits 0–9 once instead of recomputing per digit.

PART 3 — NUMBER PROPERTIES

Q17. Palindrome Number

Problem: Check whether a non-negative integer reads the same forwards and backwards. `1221` → Palindrome.

Concepts: digit reversal (Q10), comparing to the original, widening type.

How to think: A number is a palindrome if it equals its own reversal. Reverse it with the digit toolkit and compare with a saved copy of the original. Negative numbers are not palindromes (the minus sign has no mirror).

Solution

```cpp
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;

    if (n < 0) {
        cout << "Not a palindrome\n";
        return 0;
    }

    int original = n;
    long long reversed = 0;                  // reversal can exceed INT_MAX
    while (n > 0) {
        reversed = reversed * 10 + n % 10;
        n /= 10;
    }

    cout << (reversed == original ? "Palindrome" : "Not a palindrome") << '\n';
    return 0;
}
```

Dry run (1221): reversed builds 1 → 12 → 122 → 1221 == original → **Palindrome**.

Input	Output	Note
`1221`	Palindrome	
`123`	Not a palindrome	
`0`, `7`	Palindrome	single digit
`10`	Not a palindrome	reverses to `1`
`1999999999`	Not a palindrome	reversal is 9999999991, needs `long long`

Complexity: $O(\log_{10} n)$ time, O(1) space.

Follow-ups

- *Avoid overflow without* `long long`*?* Reverse only **half** the digits: while `n > reversed`, move digits across; then compare `n == reversed` or `n == reversed / 10`. Numbers ending in 0 (except 0) are never palindromes.
- *String approach?* Convert to a string and compare with its reverse (the problem here forbids it, but it's a valid alternative in other settings).

Q18. Greatest Common Divisor (GCD)

Problem: Find the GCD of two non-negative integers. Try the Euclidean algorithm. `48 18` → `6`.

Concepts: Euclid's algorithm, loop invariant, `%`.

How to think

1. **Brute force:** try every number from `min(a, b)` down to 1. O(min(a, b)).
2. **Euclid's insight:** any common divisor of `a` and `b` also divides `a - b`, hence also `a mod b`. So `gcd(a, b) = gcd(b, a mod b)`. The numbers shrink very fast, and when `b = 0` the answer is `a`.

Solution

```
#include <iostream>
using namespace std;

int main() {
    long long a, b;
    cin >> a >> b;

    if (a < 0 || b < 0) {
        cout << "Enter non-negative numbers\n";
        return 0;
    }

    while (b != 0) {
        long long r = a % b;
        a = b;
        b = r;
    }
    cout << a << '\n';
    return 0;
}
```

Dry run (48, 18):

a	b	a % b
48	18	12
18	12	6
12	6	0
6	0	stop, answer **6**

Input	Output
`48 18`	`6`
`7 0`	`7`
`0 0`	`0` (convention)
`17 13`	`1` (coprime)
`18 48`	`6` (first step swaps them)

Complexity: O(log(min(a, b))). The worst case is two consecutive Fibonacci numbers. Space O(1).

Pitfalls: dividing by zero if you compute `a % b` when `b == 0`; forgetting that `gcd(a, 0) = a`.

Follow-ups

- *Library version?* `std::gcd(a, b)` in `<numeric>` (C++17).
- *Extended Euclid?* Also returns `x, y` with `ax + by = gcd(a, b)`, used for modular inverses.
- *GCD of many numbers?* Fold: `g = gcd(g, x)`.

Q19. Least Common Multiple (LCM)

Problem: Find the LCM of two integers. Handle zero and reduce overflow risk. `4 6` → `12`.

Concepts: `gcd · lcm = a · b`, divide before multiplying, overflow pre-check.

How to think: Since `gcd(a, b) × lcm(a, b) = a × b`, we get `lcm = a / gcd × b`. Dividing **first** keeps the intermediate value small: `a * b / g` can overflow even when the final answer fits. Handle zero separately (`lcm(a, 0) = 0`, and it avoids dividing by a zero gcd).

Solution

```
#include <iostream>
#include <climits>
using namespace std;

long long gcd(long long a, long long b) {
    while (b != 0) {
        long long r = a % b;
        a = b;
        b = r;
    }
    return a;
}

int main() {
    long long a, b;
    cin >> a >> b;

    if (a < 0 || b < 0) {
        cout << "Enter non-negative numbers\n";
        return 0;
    }
    if (a == 0 || b == 0) {
        cout << 0 << '\n';
        return 0;
    }

    long long g = gcd(a, b);
    long long reduced = a / g;              // exact division, no loss

    if (reduced > LLONG_MAX / b) {          // reduced * b would overflow
        cout << "Overflow\n";
        return 0;
    }
    cout << reduced * b << '\n';
    return 0;
}
```

Dry run (4, 6): `g = 2`, `reduced = 2`, `2 * 6 = 12`.

Input	Output
`4 6`	`12`
`7 13`	`91`
`5 0`	`0`
`9000000000000000000 9000000000000000001`	`Overflow`

Complexity: O(log(min(a, b))).

Pitfalls: forgetting `#include <climits>` for `LLONG_MAX`; naming your function `gcd` and getting an ambiguity with `std::gcd` when `<numeric>` is included together with `using namespace std;` (rename it `myGcd` if that happens).

Follow-ups

- *Why divide first?* `a / g` is exact (g divides a), so you lose nothing and keep numbers smaller.
- *LCM of a list?* Fold with `lcm = lcm / gcd(lcm, x) * x`; it grows fast, so overflow checks matter.

Q20. Perfect Number

Problem: Check whether a positive integer equals the sum of its proper divisors. `6 = 1 + 2 + 3`.

Concepts: divisor pairs, √n loop, avoiding double counting.

How to think: A proper divisor is a divisor smaller than `n`. Scanning `1..n-1` is O(n). Using divisor pairs `(i, n/i)` we only scan to √n and add both members of each pair. The only trap is a perfect square, where `i == n / i` must be added once.

Solution

```
#include <iostream>
using namespace std;

int main() {
    long long n;
    cin >> n;

    if (n <= 1) {                              // 1 has no proper divisors that sum to 1
        cout << "Not a perfect number\n";
        return 0;
    }

    long long sum = 1;                         // 1 divides every n > 1
    for (long long i = 2; i <= n / i; i++) {
        if (n % i == 0) {
            sum += i;
            long long other = n / i;
            if (other != i)                    // do not count a square root twice
                sum += other;
        }
    }
```

```
    cout << (sum == n ? "Perfect number" : "Not a perfect number") << '\n';
    return 0;
}
```

Dry run (28): `sum = 1`; `i = 2`: `+2 +14 = 17`; `i = 3`: no; `i = 4`: `+4 +7 = 28`; `i = 5`: `5 <= 28/5 = 5`, no divisor; `i = 6`: `6 <= 4` ? stop. `sum = 28 == n` → **Perfect**.

Input	Output
`6`, `28`, `496`, `8128`	Perfect
`12`	Not (1+2+3+4+6 = 16, abundant)
`8`	Not (1+2+4 = 7, deficient)
`1`	Not
`16`	Not (`other == i` case handled)

Complexity: O(√n) time, O(1) space.

Follow-ups

- *Known perfect numbers?* 6, 28, 496, 8128, 33550336. Every even perfect number has the form `2^(p-1) (2^p – 1)` where `2^p – 1` is a Mersenne prime (Euclid–Euler theorem).
- *Terminology:* sum of proper divisors `< n` → deficient, `= n` → perfect, `> n` → abundant.

Q21. Strong Number

Problem: Check whether a number equals the sum of the factorials of its digits. `145 = 1! + 4! + 5!`.

Concepts: digit toolkit, factorial of a single digit, lookup table.

How to think: Peel each digit and add its factorial. Digits are only 0–9, so factorials are at most `9! = 362880`; there is no overflow risk and no need for a general factorial function. A lookup table is the cleanest.

Solution

```
#include <iostream>
using namespace std;

int main() {
    const int fact[10] = {1, 1, 2, 6, 24, 120, 720, 5040, 40320, 362880};

    long long n;
    cin >> n;

    if (n <= 0) {
        cout << "Not a strong number\n";
        return 0;
    }

    long long temp = n, sum = 0;
    while (temp > 0) {
```

```
        sum += fact[temp % 10];
        temp /= 10;
    }

    cout << (sum == n ? "Strong number" : "Not a strong number") << '\n';
    return 0;
}
```

Dry run (145): `5! = 120`, `4! = 24`, `1! = 1` → `145` ✓.

Input	Output
`145`	Strong
`40585`	Strong (24 + 1 + 120 + 40320 + 120)
`1`, `2`	Strong (`1! = 1`, `2! = 2`)
`123`	Not (1 + 2 + 6 = 9)
`0`	Not (`0! = 1 ≠ 0`)

Only four strong numbers exist in base 10: `1, 2, 145, 40585`.

Complexity: O($\log_{10} n$) time, O(1) space.

Pitfalls: `0! = 1`, so the digit 0 contributes 1 (the table handles it); looping on `n > 0` makes input `0` skip the loop, so decide its answer explicitly.

Follow-ups

- *Why can't a strong number be large?* A d-digit number is at least `10^(d-1)`, but the digit-factorial sum is at most `d × 362880`. For d ≥ 8 the sum is always smaller, so none exist.

Q22. Power Without `std::pow()`

Problem: Compute `a^b` with a loop, `b ≥ 0`. Consider overflow and `0^0`.

Concepts: repeated multiplication, overflow pre-check, special cases, fast exponentiation.

How to think

1. `a^b = a × a × … × a` (b times); `a^0 = 1`.
2. **Convention:** `0^0 = 1` (empty product; consistent with the binomial theorem and `std::pow`).
3. Work with magnitudes `|a|`, check overflow before each multiply, and apply the sign at the end: negative only if `a < 0` and `b` is odd.
4. If `|a| ≤ 1` the value never grows, so skip the loop (otherwise `1^2000000000` would loop 2 billion times). If `|a| ≥ 2` it doubles at least each step, so overflow ends the loop within about 63 iterations.

Solution

```
#include <iostream>
#include <climits>
using namespace std;
```

```
int main() {
    long long a;
    int b;
    cin >> a >> b;

    if (b < 0) {
        cout << "Exponent must be non-negative\n";
        return 0;
    }
    if (a == LLONG_MIN) {                       // -a would overflow
        cout << "Input too large\n";
        return 0;
    }

    long long absA = (a < 0) ? -a : a;
    long long mag = 1;

    if (b == 0)          mag = 1;               // includes 0^0 = 1
    else if (absA == 0)  mag = 0;
    else if (absA == 1)  mag = 1;
    else {
        for (int i = 0; i < b; i++) {
            if (mag > LLONG_MAX / absA) {       // mag * absA would overflow
                cout << "Overflow\n";
                return 0;
            }
            mag *= absA;
        }
    }

    bool negative = (a < 0) && (b % 2 == 1);
    cout << (negative ? -mag : mag) << '\n';
    return 0;
}
```

Input (a b)	Output
`2 10`	`1024`
`5 0`	`1`
`0 0`	`1`
`0 5`	`0`
`-2 3`	`-8`
`2 62`	`4611686018427387904`
`2 63`	`Overflow` (note: -2^{63} exists, but 2^{63} itself does not fit)

Complexity: O(b), but at most about 63 iterations when `|a| ≥ 2` because overflow stops it.

Bonus — fast exponentiation (binary exponentiation), O(log b)

```
long long power(long long base, long long exp) {
    long long result = 1;
    while (exp > 0) {
        if (exp & 1) result *= base;   // current bit is 1: multiply in
        base *= base;                  // square the base
        exp >>= 1;                     // next bit
    }
    return result;
}
```

Trace for `3^13`: `13 = 1101₂` → `3¹ · 3⁴ · 3⁸ = 3 · 81 · 6561 = 1,594,323`. (Add overflow checks or use modular arithmetic for real use.)

Pitfalls: using `double` and printing `1.04858e+06` for `2^20`; starting the result at 0; ignoring `0^0`.

Follow-ups

- *Why is fast exponentiation correct?* `a^b = (a²)^(b/2)` for even `b`, and `a · (a²)^((b-1)/2)` for odd `b`; the binary digits of `b` choose which squares to multiply.
- *Modular version?* Take `% mod` after every multiplication (used in cryptography and combinatorics).
- *Negative exponents or real bases?* Use `double` and `1.0 / a^|b|`, and print with `setprecision`.

Q23. Count Even and Odd Digits

Problem: Count the even and odd digits of an integer. `123456` → 3 even, 3 odd.

Concepts: digit toolkit, parity test, `do-while`.

How to think: Peel each digit and classify it with `% 2`. Use `do-while` so `0` counts as one even digit. Testing `digit % 2 == 0` for evenness works for negatives too, because `-3 % 2 = -1 ≠ 0` falls into the odd branch.

Solution

```
#include <iostream>
using namespace std;

int main() {
    long long n;
    cin >> n;

    int evenCount = 0, oddCount = 0;
    do {
        int digit = static_cast<int>(n % 10);
        if (digit % 2 == 0) evenCount++;
        else                oddCount++;
        n /= 10;
    } while (n != 0);

    cout << "Even digits: " << evenCount << '\n';
    cout << "Odd digits: " << oddCount << '\n';
    return 0;
}
```

Dry run (123456): digits 6,5,4,3,2,1 → even `6, 4, 2` = 3; odd `5, 3, 1` = 3.

Input	Output
`123456`	3 even, 3 odd
`0`	1 even, 0 odd
`-2468`	4 even, 0 odd
`1357`	0 even, 4 odd

Complexity: O($\log_{10}$ n) time, O(1) space.

Follow-ups

- *Count a specific digit?* Compare `digit == target` instead.
- *Total digits check:* `evenCount + oddCount` equals the digit count from Q11.

Q24. Find the First and Last Digit

Problem: Extract the first and last digits of a non-negative integer without strings. `95731` → first 9, last 1.

Concepts: `% 10`, repeated division.

How to think: The **last** digit is `n % 10`. The **first** digit is what remains after dividing by 10 until only one digit is left (`n < 10`). Use a copy so the original stays intact.

Solution

```
#include <iostream>
using namespace std;

int main() {
    long long n;
    cin >> n;

    if (n < 0) {
        cout << "Enter a non-negative number\n";
        return 0;
    }

    int last = static_cast<int>(n % 10);

    long long first = n;
    while (first >= 10)
        first /= 10;

    cout << "First digit: " << first << '\n';
    cout << "Last digit: " << last << '\n';
    return 0;
}
```

Dry run (95731): `last = 1`. `first`: 95731 → 9573 → 957 → 95 → **9**.

Input	First	Last
`95731`	9	1
`7`	7	7
`0`	0	0
`1000`	1	0

Complexity: O($\log_{10}$ n) time, O(1) space.

Pitfalls: destroying `n` before computing `last`; using `pow(10, digits-1)` (floating-point rounding).

Follow-ups

- *Swap the first and last digits of a number?* `n - first * p - last + last * p + first`, where `p = 10^(digits-1)`; compute `p` while stripping digits.
- *Why is the loop condition* `>= 10` *and not* `> 0`*?* `> 0` would strip the last remaining digit and return 0.

PART 4 — NUMBER CHALLENGES AND PATTERNS

Q25. Swap Two Numbers

Problem: Swap two variables, first with a temporary variable, then without one.

Concepts: references (C6), arithmetic trick, XOR trick (C9), `std::swap`.

How to think: You cannot do `a = b; b = a;` because the first assignment destroys `a`. Either save one value (temp), or encode both values into one variable and decode.

Solution

```
#include <iostream>
#include <utility>                // std::swap
using namespace std;

void swapWithTemp(int &a, int &b) {          // method 1: clear and safe
    int temp = a;
    a = b;
    b = temp;
}

void swapArithmetic(int &a, int &b) {        // method 2: no temp
    a = a + b;                               // a holds the sum
    b = a - b;                               // (a+b) - b = old a
    a = a - b;                               // (a+b) - old a = old b
}

void swapXor(int &a, int &b) {               // method 3: no temp
    if (&a == &b) return;                    // x ^ x == 0 would erase the value
    a ^= b;
    b ^= a;
    a ^= b;
```

```
}

int main() {
    int a, b;
    cin >> a >> b;

    int x = a, y = b;
    swapWithTemp(x, y);
    cout << "Temp:       " << x << ' ' << y << '\n';

    x = a; y = b;
    swapArithmetic(x, y);
    cout << "Arithmetic: " << x << ' ' << y << '\n';

    x = a; y = b;
    swapXor(x, y);
    cout << "XOR:        " << x << ' ' << y << '\n';

    x = a; y = b;
    swap(x, y);
    cout << "std::swap:  " << x << ' ' << y << '\n';
    return 0;
}
```

Dry run (arithmetic, a = 5, b = 8): a = 13 → b = 13 − 8 = 5 → a = 13 − 5 = 8 ✓ **Dry run (XOR, a = 5 = 101_2, b = 3 = 011_2):** a = 110 → b = 110 ^ 011 = 101 (5) → a = 110 ^ 101 = 011 (3) ✓

Method	Pros	Cons
Temp variable	clear, works for any type	one extra variable
Arithmetic	no temp	`a + b` can overflow (signed overflow is UB)
XOR	no temp, no overflow	integers only; fails if both names refer to the same variable
`std::swap`	idiomatic, optimized, any type	none

Complexity: O(1).

Pitfalls: forgetting `&` (parameters by value swap local copies only); XOR swap on the same object zeroing it.

Follow-ups

- *Which would you use in production?* `std::swap`. The "no temp" tricks are curiosities and often slower on modern CPUs, because they create dependencies between instructions.

Q26. Optimized Prime Check (√n)

Problem: Improve the prime checker to test divisors only up to √n. Explain why it works.

Concepts: √n bound proof, 6k ± 1, overflow-safe condition (see Q12 for the code).

How to think (the proof interviewers want)

Suppose `n` is composite, so `n = a × b` with `1 < a ≤ b < n`.

- If both `a` and `b` were greater than √n, then `a × b > √n × √n = n`. Contradiction.
- So the smaller factor `a` satisfies `a ≤ √n`.
- Therefore, if `n` has no divisor in `[2, √n]`, it has none at all and is prime.

Solution

```
#include <iostream>
using namespace std;

bool isPrime(int n) {
    if (n <= 1) return false;
    if (n <= 3) return true;
    if (n % 2 == 0 || n % 3 == 0) return false;
    for (int i = 5; i <= n / i; i += 6)
        if (n % i == 0 || n % (i + 2) == 0)
            return false;
    return true;
}

int main() {
    int n;
    cin >> n;
    cout << (isPrime(n) ? "Prime" : "Not prime") << '\n';
    return 0;
}
```

Example (n = 36, √n = 6): divisor pairs `(2, 18) (3, 12) (4, 9) (6, 6)`: every pair has a member ≤ 6, so the very first test finds a factor.

Speed comparison (n ≈ 10^9):

Strategy	Approximate checks
all `i` in `[2, n-1]`	1,000,000,000
`i` up to √n	31,623
√n with step 6	~10,500

Complexity: O(√n).

Follow-ups

- `i <= n / i` *vs* `i * i <= n`*?* Equivalent, but the first cannot overflow.
- *Even faster for many numbers?* Sieve of Eratosthenes (Q27) or a deterministic Miller–Rabin test for huge numbers.

Q27. Print All Primes in a Range

Problem: Print every prime between two integers. `10 30` → `11 13 17 19 23 29`.

Concepts: reusing a function, safe loop bounds, Sieve of Eratosthenes.

How to think: For each number in `[start, end]`, call `isPrime`. Cost: `(end − start) × √end`. The unusual loop shape protects against `end == INT_MAX`, where `n <= end; n++` would overflow and never terminate.

Solution

```
#include <iostream>
using namespace std;

bool isPrime(int n) {
    if (n <= 1) return false;
    for (int i = 2; i <= n / i; i++)
        if (n % i == 0) return false;
    return true;
}

int main() {
    int start, end;
    cin >> start >> end;

    if (start > end) {
        cout << "Invalid range\n";
        return 0;
    }

    bool first = true;
    for (int n = start;; n++) {                // safe even when end == INT_MAX
        if (isPrime(n)) {
            if (!first) cout << ' ';
            cout << n;
            first = false;
        }
        if (n == end) break;
    }
    cout << '\n';
    return 0;
}
```

Input	Output
`10 30`	`11 13 17 19 23 29`
`1 10`	`2 3 5 7`
`24 28`	(empty line)
`30 10`	`Invalid range`
`2147483600 2147483647`	`2147483629 2147483647`

Complexity: O((end − start) · √end).

Better for big ranges — Sieve of Eratosthenes, O(N log log N)

```
#include <vector>
// primes up to N
vector<bool> isP(N + 1, true);
isP[0] = isP[1] = false;
for (long long i = 2; i * i <= N; i++)
    if (isP[i])
        for (long long j = i * i; j <= N; j += i)   // start at i*i: smaller multiples already cro
            isP[j] = false;
```

Idea: cross out the multiples of each prime; whatever stays un-crossed is prime.

Follow-ups

- *Why start the inner loop at* `i * i`*?* Any smaller multiple `i × k` with `k < i` was already crossed out by a smaller prime factor.
- *Memory?* `vector<bool>` uses one bit per number.

Q28. Prime Factorization

Problem: Print the prime factors of a positive integer, including repeats. `12` → `2 2 3`.

Concepts: trial division, dividing out repeatedly, leftover prime.

How to think: Try divisors `i = 2, 3, 4, …`. Whenever `i` divides `n`, print it and divide it out, **repeating** while it still divides. Composite `i` never divide `n` because their prime factors were already removed. Stop when `i > √n`; whatever remains (> 1) is a single prime larger than √n.

Solution

```
#include <iostream>
using namespace std;

int main() {
    long long n;
    cin >> n;

    if (n < 2) {
        cout << "Enter a number greater than 1\n";
        return 0;
    }

    bool first = true;
    auto printFactor = [&](long long f) {      // separators only BETWEEN factors
        if (!first) cout << ' ';
        cout << f;
        first = false;
    };

    for (long long i = 2; i <= n / i; i++) {
        while (n % i == 0) {                   // repeated factors
            printFactor(i);
            n /= i;
        }
    }
```

```
    if (n > 1)                                  // leftover prime > sqrt(original n)
        printFactor(n);

    cout << '\n';
    return 0;
}
```

Dry run (84):

i	divides?	printed	n after
2	yes, yes	2, 2	21
3	yes	3	7
4	`4 <= 7/4 = 1`? no, loop ends		7
after loop	`n = 7 > 1`	7	

Output: `2 2 3 7`.

Input	Output
`12`	`2 2 3`
`8`	`2 2 2`
`97`	`97`
`2`	`2`
`1`	message

Complexity: O(√n) worst case (when `n` is prime or a product of two large primes).

Pitfalls: forgetting the leftover `n > 1`; using `if` instead of `while` (loses repeated factors); trailing space.

Follow-ups

- *Why only check up to √n?* After removing all prime factors ≤ √n, at most one prime factor > √n can remain (two such factors would multiply to more than n).
- *Faster for many queries?* Precompute the smallest prime factor for all numbers with a sieve; each factorization then takes O(log n).
- *Count divisors from factorization:* if `n = p₁^e₁ · p₂^e₂ …`, the divisor count is `(e₁+1)(e₂+1)…`.

Q29. Convert Decimal to Binary

Problem: Convert a non-negative integer to binary without a conversion library. `13` → `1101`.

Concepts: base conversion by repeated division, reversing, `char` arithmetic (C10).

How to think: Dividing by 2 repeatedly, the **remainders** are the binary digits from least significant to most significant. So collect them and reverse. Handle `0` separately, since the loop would produce an empty string.

Solution

```
#include <iostream>
#include <string>
#include <algorithm>
using namespace std;

int main() {
    long long n;
    cin >> n;

    if (n < 0) {
        cout << "Enter a non-negative number\n";
        return 0;
    }

    string binary;
    if (n == 0) {
        binary = "0";
    } else {
        while (n > 0) {
            binary += char('0' + n % 2);       // next bit, least significant first
            n /= 2;
        }
        reverse(binary.begin(), binary.end()); // bits came out backwards
    }

    cout << binary << '\n';
    return 0;
}
```

Dry run (13):

n	n % 2	bits so far
13	1	`1`
6	0	`10`
3	1	`101`
1	1	`1011`

Reverse → `1101` (8 + 4 + 0 + 1 = 13 ✓).

Input	Output
`10`	`1010`
`0`	`0`
`1`	`1`
`255`	`11111111`

Complexity: O($\log_2 n$) time and space.

Alternative (bit scanning, no reverse): loop `for (int i = 62; i >= 0; i--)` printing `(n >> i) & 1`, skipping leading zeros.

Follow-ups

- *Convert to any base up to 36?* Replace `2` with the base; map digits ≥ 10 to letters.
- *Negative numbers?* Use two's complement (a fixed width such as 32 bits).

Q30. Convert Binary to Decimal

Problem: Convert a binary number (given as digits) to decimal. Validate that only `0` and `1` appear. `1101` → `13`.

Concepts: Horner's method, string input, validation, overflow pre-check.

How to think: Reading left to right, each new bit shifts the value left by one position and adds the bit: `value = value × 2 + bit` (Horner's method). Read the input as a **string**; a long binary number cannot be stored in an integer. Validate every character and check overflow before each step.

Solution

```
#include <iostream>
#include <string>
#include <climits>
using namespace std;

int main() {
    string binary;
    cin >> binary;

    if (binary.empty()) {
        cout << "Invalid binary number\n";
        return 0;
    }

    long long decimal = 0;
    for (char bit : binary) {
        if (bit != '0' && bit != '1') {        // validation
            cout << "Invalid binary number\n";
            return 0;
        }
        int digit = bit - '0';

        if (decimal > (LLONG_MAX - digit) / 2) { // decimal*2 + digit would overflow
            cout << "Number is too large\n";
            return 0;
        }
        decimal = decimal * 2 + digit;
    }

    cout << decimal << '\n';
    return 0;
}
```

Dry run (1101): 0 → `0·2+1 = 1` → `1·2+1 = 3` → `3·2+0 = 6` → `6·2+1 = 13`.

Input	Output
`1101`	`13`
`0`	`0`
`00101`	`5` (leading zeros are fine)
`1021`	`Invalid binary number`

Complexity: O(length) time, O(1) extra space.

Why the overflow condition is right: `decimal * 2 + digit <= LLONG_MAX` ⟺ `decimal <= (LLONG_MAX - digit) / 2` (integer division floors, which is exactly what we need).

Follow-ups

- *Same idea in any base?* `value = value × base + digit`.
- *Alternative:* sum `bit × 2^position` going right to left.

Q31. Power of Two

Problem: Decide whether a positive integer is a power of two. Bonus: use a bitwise operation.

Concepts: binary representation, `n & (n - 1)` (C9).

How to think: A power of two has **exactly one** `1` bit (`8 = 1000₂`). Subtracting 1 turns that bit into 0 and every lower bit into 1 (`7 = 0111₂`), so `n & (n - 1)` is `0`. For any other number, at least one higher set bit survives. Guard `n > 0`, because `0 & -1 == 0` would wrongly pass.

Solution

```
#include <iostream>
using namespace std;

int main() {
    long long n;
    cin >> n;

    bool result = n > 0 && (n & (n - 1)) == 0;
    cout << (result ? "Power of two" : "Not a power of two") << '\n';
    return 0;
}
```

```
n = 8 = 1000        n = 6 = 0110
n-1   = 0111        n-1   = 0101
n&n-1 = 0000 → yes  n&n-1 = 0100 → no
```

Input	Output
`1`	Power of two (2^0)
`16`	Power of two

Input	Output
`18`	Not
`0`, `-8`	Not (`n > 0` guard)

Complexity: O(1).

Loop alternative: `while (n % 2 == 0) n /= 2; return n == 1;` — O(log n).

Follow-ups

- *Count set bits?* Kernighan: `while (n) { n &= n - 1; count++; }` runs once per set bit.
- *Find the next power of two ≥ n?* Smear bits right then add 1, or use `bit_ceil` (C++20).

Q32. Digital Root

Problem: Repeatedly sum the digits until one digit remains. `9875` → `2`.

Concepts: digit sum loop, modulo-9 shortcut.

How to think: Do exactly what the statement says: `9875 → 29 → 11 → 2`. Mathematically, a number is congruent to its digit sum modulo 9, so the digital root equals `n mod 9`, with `9` standing in for multiples of 9 (and `0` for 0): `1 + (n − 1) % 9` for `n > 0`.

Solution

```
#include <iostream>
using namespace std;

int main() {
    long long n;
    cin >> n;

    if (n < 0) {
        cout << "Enter a non-negative number\n";
        return 0;
    }

    while (n >= 10) {                    // repeat until a single digit
        long long sum = 0;
        while (n > 0) {
            sum += n % 10;
            n /= 10;
        }
        n = sum;
    }

    cout << n << '\n';
    return 0;
}
```

O(1) formula

```
long long root = (n == 0) ? 0 : 1 + (n - 1) % 9;
```

Check: `1 + (9875 − 1) % 9 = 1 + 9874 % 9 = 1 + 1 = 2` ✓

Input	Output
`9875`	`2`
`7`	`7`
`0`	`0`
`99`	`9`

Complexity: loop O(log n) per pass and very few passes; formula O(1).

Follow-ups

- *Why does mod 9 work?* `10 ≡ 1 (mod 9)`, so each digit contributes `digit × 1`; the digit sum has the same remainder as the number.
- *Use:* quick divisibility test for 3 and 9.

PATTERN PROBLEMS (Q33–Q40)

Method (recap of C11): outer loop = rows `i`; for each row decide how many **spaces** and how many **symbols**; print separators only between items; end each row with `'\n'`.

Pattern	Row `i` content
Q33 right triangle	`i` stars
Q34 inverted triangle	`n - i + 1` stars (or count `i` downward)
Q35 number triangle	numbers `1..i` (column index)
Q36 repeated numbers	the number `i`, `i` times (row index)
Q37 pyramid	`n - i` spaces, `2i - 1` stars
Q38 diamond	pyramid up to the middle, then mirrored
Q39 Floyd	the next `i` numbers from a running counter
Q40 hollow square	star on the border, space inside

Q33. Right-Angled Star Triangle

Problem: For `n = 4` print rows of `1, 2, 3, 4` stars.

Solution

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;

    for (int i = 1; i <= n; i++) {          // row i
        for (int j = 1; j <= i; j++)        // i stars
            cout << '*';
        cout << '\n';
    }
    return 0;
}
```

Output (n = 4):

```
*
**
***
****
```

Dry run: `i = 1` → inner loop runs once → `*`; `i = 2` → twice → `**`; …

Edge cases: `n = 0` or negative prints nothing (loop never runs); `n = 1` prints `*`.

Complexity: O(n^2) time (number of characters printed is `n(n+1)/2`), O(1) space.

Pitfalls: printing `"* "` adds spaces the expected output doesn't have; forgetting the `'\n'`.

Q34. Inverted Star Triangle

Problem: For `n = 4` print rows of `4, 3, 2, 1` stars.

Solution

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;

    for (int i = n; i >= 1; i--) {          // counts DOWN
        for (int j = 1; j <= i; j++)
            cout << '*';
        cout << '\n';
    }
    return 0;
}
```

Output (n = 4):

```
****
***
**
*
```

How to think: Same inner loop as Q33; only the outer loop direction changes. Equivalent form: row `r = 1..n` prints `n - r + 1` stars.

Complexity: $O(n^2)$.

Q35. Number Triangle

Problem: For `n = 4` print `1`, `12`, `123`, `1234`.

Solution

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++)
            cout << j;                      // the COLUMN number
        cout << '\n';
    }
    return 0;
}
```

Output (n = 4):

```
1
12
123
1234
```

How to think: Row `i` has `i` columns, and the value in column `j` is `j`.

Pitfalls / follow-up: for `n ≥ 10` the numbers run together (`12345678910`). In an interview, ask whether to put a separator; then print `if (j > 1) cout << ' ';` before `j`.

Q36. Repeated-Number Triangle

Problem: For `n = 4` print `1`, `22`, `333`, `4444`.

Solution

```
#include <iostream>
using namespace std;
```

```
int main() {
    int n;
    cin >> n;

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++)
            cout << i;                      // the ROW number
        cout << '\n';
    }
    return 0;
}
```

Output (n = 4):

```
1
22
333
4444
```

Q35 vs Q36 (a favorite interview pair): same loops. Q35 prints the **column** `j`; Q36 prints the **row** `i`.

Q37. Pyramid Pattern

Problem: For `n = 4` print a centered pyramid with `1, 3, 5, 7` stars.

How to think: Make a table and find formulas.

Row i	Leading spaces `n - i`	Stars `2i - 1`
1	3	1
2	2	3
3	1	5
4	0	7

Only **leading** spaces are printed (never trailing), so lines match the expected output exactly.

Solution

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;

    for (int i = 1; i <= n; i++) {
        for (int s = 0; s < n - i; s++)         // leading spaces
            cout << ' ';
        for (int j = 0; j < 2 * i - 1; j++)     // stars: 1, 3, 5, 7, ...
            cout << '*';
        cout << '\n';
    }
```

```
    return 0;
}
```

Output (n = 4):

```
   *
  ***
 *****
*******
```

Complexity: O(n^2).

Follow-up: *Inverted pyramid?* Loop `i` from `n` down to `1`.

Q38. Diamond Pattern

Problem: Print a symmetric diamond of stars for a given **height**. Define whether the height is odd or even and handle it consistently.

How to think

1. A diamond with a single-star tip and a single widest row has an **odd** height `h`. That is the definition I'll state in the interview: "*Height must be odd; if an even number is given, I round it up to the next odd number.*"
2. The middle row is `mid = (h + 1) / 2`.
3. For row `i`, let `k = i` for the top half and `k = h - i + 1` for the bottom half (the distance from the nearest tip). Then spaces `= mid - k` and stars `= 2k - 1`, exactly like the pyramid.

Solution

```
#include <iostream>
using namespace std;

int main() {
    int h;
    cin >> h;

    if (h <= 0) return 0;

    if (h % 2 == 0) {                          // even -> round up to odd
        h++;
        cerr << "Even height: using " << h << " instead\n";   // stderr keeps stdout clean
    }

    int mid = (h + 1) / 2;                     // row with the widest line

    for (int i = 1; i <= h; i++) {
        int k = (i <= mid) ? i : h - i + 1;    // distance from the nearest tip

        for (int s = 0; s < mid - k; s++)
            cout << ' ';
        for (int j = 0; j < 2 * k - 1; j++)
            cout << '*';
        cout << '\n';
```

```
    }
    return 0;
}
```

Output (h = 7):

```
   *
  ***
 *****
*******
 *****
  ***
   *
```

Row i	k	spaces `mid-k`	stars `2k-1`
1	1	3	1
2	2	2	3
3	3	1	5
4	4	0	7
5	3	1	5
6	2	2	3
7	1	3	1

Input	Behavior
`1`	`*`
`5`	5-row diamond
`6`	treated as `7`
`0`, negative	prints nothing

Variant with spaced stars (`* * *` style)

```
for (int s = 0; s < 2 * (mid - k); s++) cout << ' ';
for (int j = 0; j < 2 * k - 1; j++) {
    if (j > 0) cout << ' ';                    // space BETWEEN stars only
    cout << '*';
}
```

Complexity: O(h^2).

Follow-ups

- *Hollow diamond?* Print a star only at the first and last positions of each row.

- *Alternative implementation?* Print the top half (like Q37 up to `mid`) and the bottom half with `i` from `mid - 1` down to 1; both are fine, but the single-loop form avoids duplicated code.

Q39. Floyd's Triangle

Problem: For `n = 4` print consecutive numbers in a triangle: `1 / 2 3 / 4 5 6 / 7 8 9 10`.

How to think: Row `i` has `i` numbers. The numbers are simply `1, 2, 3, …` taken in order, so use **one counter that never resets**.

Solution

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;

    long long number = 1;                       // running counter across rows
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            if (j > 1) cout << ' ';             // separator only between numbers
            cout << number++;
        }
        cout << '\n';
    }
    return 0;
}
```

Output (n = 4):

```
1
2 3
4 5 6
7 8 9 10
```

Useful formulas: total numbers `n(n+1)/2`; last number of row `i` is `i(i+1)/2`; first number of row `i` is `i(i−1)/2 + 1`.

Pattern	What row `i` prints
Q35	`1 … i` (restarts every row)
Q36	`i` repeated `i` times
Q39	the next `i` values of a global counter

Complexity: $O(n^2)$.

Follow-up: *Why* `long long`*?* For large `n` the counter reaches `n(n+1)/2`, which exceeds `int` once `n > 65535`.

Q40. Hollow Square Pattern

Problem: For `n = 4` print a square whose border is stars and whose inside is empty.

```
****
*  *
*  *
****
```

How to think: Every cell is on a grid `(i, j)`. It is a border cell if it is in the **first or last row** or the **first or last column**. Print `*` for border cells and a space inside. Each cell is exactly **one character** so the width is `n`.

Solution

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;

    for (int i = 1; i <= n; i++) {              // row
        for (int j = 1; j <= n; j++) {          // column
            if (i == 1 || i == n || j == 1 || j == n)
                cout << '*';
            else
                cout << ' ';
        }
        cout << '\n';
    }
    return 0;
}
```

n	Output
`1`	`*`
`2`	`**` / `**`
`3`	`***` / `* *` / `***`
`4`	the square above

Complexity: O(n^2).

Follow-ups (same trick, different condition)

- Add `|| i == j` → hollow square with a main diagonal.
- Add `|| i == j || i + j == n + 1` → hollow square with an **X**.
- Always print `*` → filled square.

PART 5 — INTERVIEW Q&A, CHECKLISTS, CONCEPT MAP

5.1 How to run a coding interview (5 steps)

1. **Understand:** restate the problem; ask about input range, negatives, zero, invalid input, output format.
2. **Examples:** write one normal case and two edge cases *before* coding.
3. **Plan:** say the brute force first with its complexity, then the improvement.
4. **Code:** small functions, meaningful names, no clever tricks you cannot explain.
5. **Test:** dry-run an example out loud, then run the edge cases.

5.2 Rapid-fire conceptual questions

Q: What is undefined behavior? Give two examples from this handbook. Behavior the standard does not define; the program may crash, give wrong results, or appear to work. Examples: signed integer overflow (`INT_MAX + 1`, Q2/Q13), dividing by zero (Q6), reading an uninitialized variable.

Q: `int` vs `long long`: when do you switch? When intermediate or final values can exceed about 2.1 × 10^9 (sums, products, factorials, reversed numbers). In contests and interviews, defaulting to `long long` for arithmetic is a safe habit.

Q: How do you detect overflow *before* it happens? Addition: `a > MAX - b`. Multiplication: `a > MAX / b`. Or compute in a wider type and range-check (Q10, Q13, Q14, Q19, Q22, Q30).

Q: Result of `-7 / 2` and `-7 % 2`? `-3` and `-1` (division truncates toward zero; the remainder takes the sign of the dividend).

Q: `while` vs `do-while`? `while` may run zero times; `do-while` runs at least once. Use `do-while` whenever the input `0` must still produce an iteration (Q11, Q15, Q23).

Q: Why is checking up to √n enough for primality? A composite `n = a·b` has a factor ≤ √n, because two factors > √n would multiply to more than `n` (Q26).

Q: Why write `i <= n / i` instead of `i * i <= n`? `i * i` can overflow near `INT_MAX`; the division form cannot (Q12, Q20, Q28).

Q: Euclid's algorithm: why does it work and how fast is it? Any common divisor of `a` and `b` divides `a mod b`, so `gcd(a, b) = gcd(b, a mod b)`. It finishes in O(log min(a, b)) steps; the worst case is consecutive Fibonacci numbers (Q18).

Q: How is LCM computed from GCD? `lcm = a / gcd(a, b) * b`; divide first to limit overflow (Q19).

Q: What does `n & (n - 1)` do? Clears the lowest set bit. Zero result ⇒ `n` has at most one set bit ⇒ power of two for `n > 0` (Q31). Repeating it counts the set bits.

Q: `endl` vs `'\n'`? `endl` = newline + flush. Prefer `'\n'` unless you need to flush.

Q: Pass by value vs by reference? By value copies the argument; by reference (`T&`) operates on the caller's variable (Q10, Q25).

Q: Why avoid `std::pow` for integer results? It returns `double`; large or exact results can be off by rounding (e.g. `124.99999` truncating to 124). Use an integer loop (Q16, Q22).

Q: Recursion vs iteration for Fibonacci/factorial? Iteration is O(1) space and cannot overflow the call stack. Naive recursive Fibonacci is O(2^n).

Q: What is a loop invariant? A statement that is true before and after every iteration, used to prove correctness (Euclid: "the gcd of the current pair equals the gcd of the original pair").

Q: How do you make the output match exactly? No prompts, print separators only *between* items, exactly one newline per line, and exact words (`Hello, World!`).

Q: Is 1 prime? Is 0 a palindrome? Is 0 an Armstrong number? 1 is not prime. Single digits and `0` are palindromes. `0` and all single digits are Armstrong numbers (`d¹ = d`).

5.3 Mistakes to avoid (found while reviewing these exact problems)

Mistake	Where it bites	Prevention
Wrong text / punctuation	Q1 (`Hello world!`)	copy the sample output exactly
`int` for sums or products	Q2, Q9, Q13	use `long long` and pre-checks
Reading input in the wrong order	Q6	follow the sample input format
Formula instead of the requested loop	Q9	show the loop, mention the formula as optimization
`std::pow` or `double` for integer powers	Q16, Q22	integer loop; guard overflow
`n % 2 == 1` for odd	Q3, Q23	use `!= 0`
`while (n > 0)` losing input `0`	Q11, Q15, Q23	`do-while`
`i * i <= n` overflow	Q12, Q20, Q26–Q28	`i <= n / i`
Missing `<climits>`	Q10, Q13, Q14, Q19, Q22, Q30	include what you use
Trailing spaces	Q14, Q27, Q28, Q39	separator only between items
Extra spaces in patterns	Q33–Q36, Q40	print one character per cell
Incomplete requirements	Q25 (only one swap method), Q38 (odd/even undefined)	re-read the problem, tick every requirement
Infinite loop at `INT_MAX`	Q27	`for (n = start;; n++) { ...; if (n == end) break; }`

5.4 Complexity table for all 40 problems

Q	Time	Space		Q	Time	Space
1–8	O(1)	O(1)		25	O(1)	O(1)
9	O(n) / O(1)	O(1)		26	O(√n)	O(1)
10, 11, 15, 17	O(log n)	O(1)		27	O(R·√N)	O(1)
12	O(√n)	O(1)		28	O(√n)	O(1)
13, 14	O(n)	O(1)		29	O(log n)	O(log n)

Q	Time	Space		Q	Time	Space
16	$O(d^2)$	O(1)		30	O(len)	O(1)
18, 19	O(log min)	O(1)		31	O(1)	O(1)
20	$O(\sqrt{n})$	O(1)		32	O(log n) / O(1)	O(1)
21, 23, 24	O(log n)	O(1)		33–37, 39, 40	$O(n^2)$	O(1)
22	O(b) (≤ 63 steps) / O(log b)	O(1)		38	$O(h^2)$	O(1)

5.5 Concept → problems map

Concept	Problems
I/O and program structure	1, 2
Conditions and boolean logic	3, 4, 5, 6, 7
`for` loops and accumulators	8, 9, 13, 14
Digit extraction (`% 10`, `/ 10`)	10, 11, 15, 16, 17, 21, 23, 24, 32
`do-while` for the zero case	11, 15, 16, 23
Overflow handling	2, 9, 10, 13, 14, 19, 22, 30
Primes and divisors	12, 20, 26, 27, 28
GCD / LCM	18, 19
Base conversion and bits	29, 30, 31, 25
Math shortcuts (formulas)	9, 32, 39
Nested loops and patterns	8, 33–40

5.6 Pre-submit checklist

- [ ] Output text and format match the sample exactly (no prompts, no trailing spaces).
- [ ] Types are wide enough (`long long`) and overflow is checked before it happens.
- [ ] Tested: `0`, `1`, negative, maximum, equal values, repeated digits.
- [ ] Loops: no off-by-one; `do-while` where `0` must be processed.
- [ ] No division or modulo by zero.
- [ ] Complexity is stated and acceptable.
- [ ] All requirements of the problem are covered (read it once more).

5.7 Practice next (same level, to extend this handbook)

Count set bits · Fibonacci with memoization · Sieve up to 10^6 · Palindrome by reversing half the digits · Binary to octal/hex · Pascal's triangle · Pyramid of numbers · Check for automorphic and happy numbers · GCD of an array.

End of handbook.